

VIDEO 2 - QUEUE & CIRCULAR QUEUE

This script follows the Queue presentation from FIFO basics through intentional wrap-around and the live C++ demonstration.

Opening / Title Slide

Hello everyone. In this part of my BLA Lab 01, I will focus on Queue and Circular Queue data structures. I will begin with the FIFO principle and the roles of FRONT and REAR. Then I will explain Enqueue, Dequeue, overflow and underflow, the limitation of a simple linear array Queue, and why a Circular Queue provides a better way to reuse fixed array space. At the end, I will demonstrate the Circular Queue program in C++.

Queue and FIFO Principle

A Queue is a linear data structure where elements are processed in the same order in which they arrive. Unlike a Stack, a Queue uses two ends. The REAR is where new elements are inserted, and the FRONT is where elements are removed.

A Queue follows FIFO, which means First In, First Out. The first element that enters the Queue is also the first element that should leave.

Queue Example and FRONT / REAR

Suppose I add 15, then 25, and then 35. The value 15 is at the FRONT because it entered first, while 35 is at the REAR because it was added most recently. If I remove one value, 15 leaves first and 25 becomes the new FRONT.

A simple real-life example is a checkout line. The customer who arrives first normally stands at the front and is served first, while new customers join at the rear. This is the same FIFO behavior used by a Queue.

Enqueue and Dequeue

The two main Queue operations are Enqueue and Dequeue. Enqueue inserts a new element at the REAR. If the Queue contains 15, 25, and 35, then enqueueing 45 places 45 at the REAR.

Dequeue removes the element from the FRONT. If I perform Dequeue on this Queue, 15 is removed because it arrived first. Then 25 becomes the new FRONT. This clearly demonstrates the First In, First Out rule.

Other Queue Checks

Just like with a Stack, we need checks to prevent invalid operations. isEmpty checks whether the Queue has no elements, and it should be checked before Dequeue. isFull checks whether all available positions in the fixed-size array are occupied. Display lets us view the current Queue from FRONT toward REAR.

Queue Diagram

This diagram summarizes the normal Queue movement. New elements enter from the REAR using Enqueue, while elements leave from the FRONT using Dequeue. Even though the values are stored in an array, the logical order always moves from FRONT to REAR.

Queue Overflow and Underflow

Queue Overflow happens when we try to Enqueue another value but the fixed-size Queue has no available position. Queue Underflow happens when we try to Dequeue from an empty Queue.

These conditions need to be handled in the program so that we do not write outside the array or try to remove a value that does not exist.

Problem With a Linear Array Queue

Now we reach an important reason for learning Circular Queue. Suppose our array has five positions and contains 15, 25, 35, 45, and 55. If I Dequeue 15 and then 25, the first two positions become available.

However, in a simple linear implementation where REAR only moves forward, REAR has already reached the final index. Even though positions zero and one are empty, REAR cannot continue forward. This means useful array space can be wasted.

We could shift all the remaining elements toward the beginning, but repeatedly shifting values creates unnecessary work. A better solution is to use a Circular Queue.

Circular Queue and Wrap-Around

A Circular Queue still uses a normal straight array in memory, but FRONT and REAR are allowed to move around it in a circular way. When REAR reaches the last index, it can return to index zero if that position is available.

In my example, after removing 15 and 25, values 35, 45, and 55 remain. When I Enqueue 65, REAR wraps around and stores 65 at index zero. When I Enqueue 75, it uses index one. This reuses the empty positions instead of wasting them.

Logical Order and Modulo Rule

After the wrap-around, the physical array may look unusual because 65 and 75 are stored near the beginning. But the logical Queue order is still 35, 45, 55, 65, 75.

The circular movement is commonly calculated with the expression $\text{rear} + 1 \text{ modulo } \text{SIZE}$. If the size is five and rear is four, then four plus one modulo five equals zero. That sends REAR back to index zero without physically moving the entire array.

Benefits of Circular Queue

The biggest advantage of a Circular Queue is better use of fixed array space. Positions that become empty after Dequeue operations can be reused. We also avoid repeatedly shifting every remaining element.

A Circular Queue still has a fixed maximum capacity when implemented with a static array. It does not create unlimited space; it simply uses the available space more efficiently.

Live C++ Demonstration

Now I will demonstrate the Circular Queue program. The program uses a fixed-size array together with front and rear variables. It also uses isEmpty and isFull checks.

I will first Enqueue 15, 25, 35, 45, and 55. Then I will Dequeue twice, which removes 15 and 25. After that, I will Enqueue 65 and 75. This is the important part of the demonstration because REAR wraps around and reuses indexes zero and one.

Finally, I will use Display to confirm that the logical Queue order is 35, 45, 55, 65, 75 even though some of those values are physically stored at the beginning of the array.

Closing

This completes my Queue and Circular Queue explanation. The main idea I learned is that a normal Queue follows FIFO, while a Circular Queue keeps the same FIFO behavior but manages fixed array space more efficiently through wrap-around.

I would like to thank Professor Victor Govindaswamy for his guidance and for helping us understand these concepts through both theoretical explanation and practical implementation. Thank you for watching.